

# Knockout Arena

## Week 2 — UI, Polish & Publishing

Extend your Knockout Arena with a full UI flow: title screen, score display, game over sequence, difficulty selection, and a publishable build. Previous worksheet must be complete before starting.

**Worksheet: User Interface + Publishing Your Project**

**SWE402: Game Development**

### Required Self-Learning — Complete BEFORE implementing

Coroutine-based UI Animations (Advanced Extension): <https://docs.unity3d.com/Manual/Coroutines.html>

**Submission:** Push updated project to the same GitHub repository + submit WebGL or desktop build link

## Requirements Checklist

### Score UI

- ☐ A Canvas exists in the scene with Render Mode set to Screen Space - Overlay.
- ☐ A TextMeshPro text element displays "Score: 0" — anchored to a screen corner via Anchor Presets.
- ☐ A second TextMeshPro text displays the current wave number (e.g., "Wave: 1").
- ☐ GameManager has a public UpdateScore(int) method that increments a private score variable and updates scoreText.text.
- ☐ Score increases when an enemy is knocked off the arena. Different enemy types can award different point values.
- ☐ Wave text updates each time a new wave begins.
- ☐ using TMPro; is included. Score and wave text variables are typed TextMeshProUGUI and assigned via Inspector.

### Game Over & Restart

- ☐ A "Game Over" TextMeshPro element exists on the Canvas — starts deactivated via SetActive(false).
- ☐ A "Restart" Button (UI > Button - TextMeshPro) exists — also starts deactivated.
- ☐ When the player falls below the arena ( $Y < -10$ ), a GameOver() method activates both UI elements and sets isActive = false.
- ☐ GameOver integrates with your existing OnGameOver delegate/event — UI subscribes via OnEnable()/OnDisable().
- ☐ Restart button calls a public RestartGame() method that uses SceneManager.LoadScene() to reload the scene.
- ☐ using UnityEngine.SceneManagement; is included. The scene is added to Build Profiles > Scene List.
- ☐ Spawning, input, and scoring are all gated by isActive — nothing happens after Game Over.

### Title & Difficulty

- ☐ A Title Screen parent object (empty) on the Canvas groups: a title text + 3 difficulty buttons (Easy, Medium, Hard).
- ☐ A DifficultyButton script with a public int difficulty field — set per-button in Inspector (e.g., 1, 2, 3).
- ☐ Each button's OnClick() calls a SetDifficulty() method which passes the difficulty int to GameManager.StartGame(int).
- ☐ StartGame(int) sets spawnRate or enemy speed based on difficulty, deactivates the title screen, and begins the game.
- ☐ Game does NOT start automatically — Start() leaves the arena idle until a difficulty button is clicked.
- ☐ On restart, the title screen reappears (scene reload handles this).

#### Variable Attributes

- ☐ All private variables that need Inspector access use [SerializeField] instead of being made public.
- ☐ At least 2 variables use [Range(min, max)] to provide slider controls in the Inspector (e.g., speed, spawnRate).
- ☐ At least 2 variables use [Tooltip("...")] to describe their purpose on hover in the Inspector.
- ☐ Variables that are public but should not appear in the Inspector use [HideInInspector].
- ☐ No variable is public solely for Inspector access — use [SerializeField] private instead.

#### Build & Publish

- ☐ WebGL Build Support (or Windows/Mac) module is installed via Unity Hub > Installs > Add Modules.
- ☐ File > Build Profiles: correct platform selected, scene is in the Scene List.
- ☐ Player Settings configured: Product Name, Company Name, default resolution set.
- ☐ A successful build is produced (WebGL folder with index.html OR a standalone .exe/.app).
- ☐ The build runs correctly outside the Unity Editor — all gameplay, UI, and audio work as expected.
- ☐ (Optional) Uploaded to Unity Play or itch.io with a shareable link.

#### Code Quality

- ☐ All previous week requirements still function correctly — no regressions from adding UI and build steps.
- ☐ No compiler warnings or errors. Console is clean on play.
- ☐ GameManager serves as the central hub — other scripts communicate through it (or via delegates) rather than finding each other directly.
- ☐ GameObject.Find() and GetComponent() calls are cached in Start() or Awake() — never called in Update().

## Advanced Extension — Coroutine UI Animations

Use yield return null (wait one frame) inside a coroutine to animate UI over time. Same pattern as your powerup timer — but instead of waiting seconds, you step through frames and lerp a value each tick.

#### Adv: Coroutine Anim

- ☐ Game Over screen fades in over ~0.5s using a CanvasGroup component and a coroutine that lerps alpha from 0 to 1.
- ☐ The coroutine uses a while loop with Time.deltaTime accumulation and yield return null (per-frame stepping).
- ☐ At least one other UI animation exists (e.g., score text scale punch on update, title screen fade-out on start, wave text slide-in).
- ☐ All animations use coroutines — no Update()-based animation or Animator for these UI effects.

#### Hint — CanvasGroup fade pattern:

```
IEnumerator FadeIn(CanvasGroup cg, float dur) {  
    float t = 0; while (t < dur) { t += Time.deltaTime; cg.alpha = t / dur; yield return null; } }
```

Good luck, and have fun with it.